In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
df_cardio = pd.read_csv("C:\\Users\\User\\Downloads\\PROJETO CARDIO
ML\\heart.csv")
```

In [3]:
```
df_cardio
```

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	52	1	0	125	212	0	1	168	0
1	53	1	0	140	203	1	0	155	1
2	70	1	0	145	174	0	1	125	1
3	61	1	0	148	203	0	1	161	0
4	62	0	0	138	294	1	1	106	0
...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1
1021	60	1	0	125	258	0	0	141	1
1022	47	1	0	110	275	0	0	118	1
1023	50	0	0	110	254	0	0	159	0
1024	54	1	0	120	188	0	1	113	0

1025 rows × 14 columns

In [4]:
```
df_cardio.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [5]:
```
df_cardio.head()
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang
0	52	1	0	125	212	0	1	168	0
1	53	1	0	140	203	1	0	155	1
2	70	1	0	145	174	0	1	125	1
3	61	1	0	148	203	0	1	161	0
4	62	0	0	138	294	1	1	106	0

In [6]:
```
from plotly.subplots import make_subplots
import plotly.graph_objects as go
```

In [7]:
```
fig = make_subplots(rows=4, cols=1)

fig.add_trace(go.Box(x=df_cardio["age"] / 365, name="Idade"), row=1,
col=1)
fig.add_trace(go.Box(x=df_cardio["chol"], name="Colesterol"), row=2,
col=1)
fig.add_trace(go.Box(x=df_cardio["trestbps"], name="Pressão do Sangue"),
row=3, col=1)
fig.add_trace(go.Box(x=df_cardio["thalach"], name="Frequencia cardiaca
máxima"), row=4, col=1)

fig.update_layout(height=700)
fig.show()
```

In [14]:

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=2, cols=3)
fig.add_trace(go.Bar(y=df_cardio["sex"].value_counts(), x=["Feminino",
"Masculino"], name="sex"), row=1, col=1)
fig.add_trace(go.Bar(y=df_cardio["chol"].value_counts(), x=["Normal",
"Acima do Normal", "Muito acima do normal"], name="chol"), row=2, col=2)
fig.add_trace(go.Bar(y=df_cardio["fbs"].value_counts(), x=["Normal",
"Acima do Normal"], name="fbs"), row=2, col=3)


fig.update_layout(height=700)
fig.show()
```

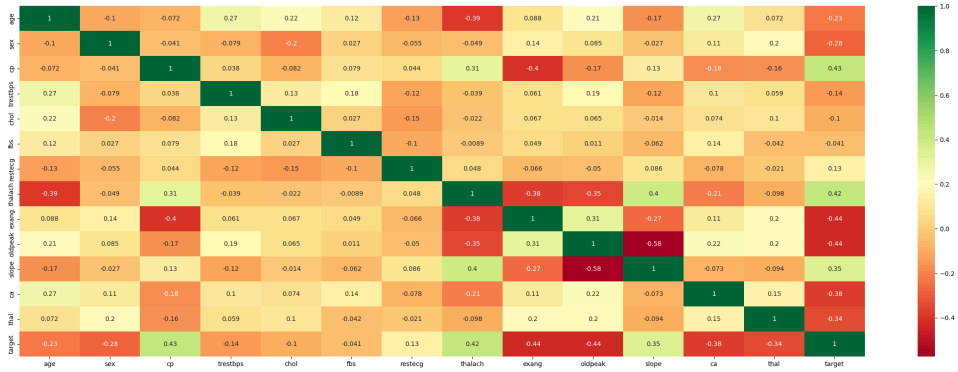
In [16]:
```
df_cardio["exang"].value_counts() /
df_cardio["exang"].value_counts().sum()
```

Out[16]:
```
exang
0    0.663415
1    0.336585
Name: count, dtype: float64
```

In [21]:
```
import seaborn as sns

fig, ax = plt.subplots(figsize=(30, 10))
sns.heatmap(df_cardio.corr(), annot=True, cmap="RdYlGn")
```

Out[21]:
```
<Axes: >
```



In [22]:
```
y = df_cardio["exang"]
x = df_cardio.loc[:, df_cardio.columns != 'exang']
```

In [23]:
```
from sklearn.model_selection import train_test_split
```

In [25]:
```
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.33, random_state=42)
```

In [26]:
```
from sklearn.ensemble import RandomForestClassifier

ml_model = RandomForestClassifier()
ml_model.fit(x_train, y_train)
```

Out[26]:
```
▾ RandomForestClassifier ⓘ ?
▸ Parameters
```

In [30]:
```
x_test.iloc[0]
```

Out[30]:
```
age          62.0
sex           0.0
cp            0.0
trestbps    124.0
chol        209.0
fbs           0.0
restecg       1.0
thalach     163.0
oldpeak       0.0
slope         2.0
ca            0.0
thal          2.0
target        1.0
Name: 527, dtype: float64
```

In [31]:
```
ml_model.predict(x_test.iloc[0].values.reshape(1, -1))
```

```
C:\Users\User\anaconda3\Lib\site-
packages\sklearn\utils\validation.py:2691: UserWarning:

X does not have valid feature names, but RandomForestClassifier
was fitted with feature names
```

Out[31]:
```
array([0])
```

In [32]:
```
y_test.iloc[0]
```

Out[32]:
```
np.int64(0)
```

In [33]:
```
from sklearn.metrics import classification_report, confusion_matrix
```

In [34]:
```
predictions = ml_model.predict(x_test)
```

In [36]:
```
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       218
           1       1.00      1.00      1.00       121

    accuracy                           1.00       339
   macro avg       1.00      1.00      1.00       339
weighted avg       1.00      1.00      1.00       339

[[218   0]
 [  0 121]]
```

In [ ]:

```

```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.